

Justification de modélisation Mexora

- Justification de modélisation
 - Requêtes-types Top-Down
 - Schéma en étoile
 - Granularité de la table de faits
 - Additivité des mesures
 - Gestion SCD
 - Cas 1 : Produit
 - Cas 2 : Client
 - Choix MySQL au lieu de PostgreSQL

Justification de modélisation

Ce document complète le rapport principal avec les éléments explicitement attendus dans le cahier de charge : besoins analytiques, granularité, additivité des mesures et cas de Slowly Changing Dimensions.

Requêtes-types Top-Down

Référence	Analyser	En fonction de	Pour
R1	Chiffre d'affaires	région, catégorie produit, mois	commandes completed et returned
R2	Quantité vendue et chiffre d'affaires	produit, trimestre, ville	produits vendus à Tanger
R3	Panier moyen	segment client, région	clients actifs dans le Data Warehouse
R4	Taux de retour	catégorie produit, région	lignes de vente chargées dans fact_sales
R5	Effet Ramadan	catégorie alimentation, période Ramadan, mois	période Ramadan 2026 approximée du 18/02/2026 au 19/03/2026

Schéma en étoile

Le Data Warehouse est organisé autour de `fact_sales`. Les dimensions principales sont :

Table	Rôle analytique
<code>dim_customer</code>	analyse par client, ville, région, tranche d'âge et segment client
<code>dim_product</code>	analyse par produit, catégorie, sous-catégorie, marque et fournisseur
<code>dim_date</code>	analyse temporelle par jour, mois, trimestre, année, week-end et Ramadan
<code>dim_region</code>	analyse géographique par ville et région
<code>dim_payment</code>	analyse par méthode et statut de paiement
<code>dim_delivery</code>	analyse par statut de livraison, transporteur, ville et région de livraison

Le cahier de charge demande une dimension livreur. Dans ce projet, les données générées ne contiennent pas d'identifiant livreur individuel. La notion est représentée par le transporteur dans `dim_delivery.shipping_company`. Un objet de compatibilité `dim_livreur` est créé dans `sql/06_reporting_views_mysql.sql` pour restituer cette lecture métier sans modifier l'ETL validé.

Granularité de la table de faits

Une ligne de `fact_sales` représente une ligne de commande nettoyée : un produit précis acheté dans une commande précise par un client donné.

Cette granularité est choisie car elle permet :

- d'agréger le chiffre d'affaires par commande, client, produit, catégorie, ville, région ou période ;
- de calculer les produits les plus vendus ;
- d'identifier les retours par produit ;
- de relier chaque vente à un paiement et une livraison.

Additivité des mesures

Mesure	Type d'additivité	Justification
<code>quantity</code>	additive	peut être sommée par produit, région ou période
<code>total_amount</code>	additive	chiffre d'affaires après remise, sommable sur toutes les dimensions
<code>amount_paid</code>	additive	montant payé réparti par ligne de commande

Mesure	Type d'additivité	Justification
refund_amount	additive	montant remboursé sommable
is_returned	additive comme compteur	SUM(is_returned) donne le nombre de lignes retournées
delivery_delay_days	semi-additive	doit être agrégé par moyenne, pas par somme
discount_rate	non-additive	un taux doit être recalculé ou moyenné selon le contexte
taux de retour	non-additive	calculé par $\text{SUM(is_returned)} / \text{COUNT(*)}$
panier moyen	non-additive	calculé par $\text{SUM(total_amount)} / \text{COUNT(DISTINCT order_id)}$

Gestion SCD

Cas 1 : Produit

Un produit peut changer de catégorie, de sous-catégorie, de fournisseur ou de statut commercial. Pour les corrections simples de libellés, le projet applique une logique SCD Type 1 : la valeur standardisée remplace la valeur brute.

Pour un changement métier réel, par exemple un produit qui passe de Accessories à Smartphones, le choix recommandé est SCD Type 2 afin de conserver l'historique analytique. Le schéma MySQL ajoute donc les colonnes `date_debut`, `date_fin` et `est_actif` dans `dim_product`, ce qui rend l'extension SCD Type 2 possible.

Cas 2 : Client

Un client peut changer de ville, de région ou de segment. Les corrections de qualité sur les villes et régions sont traitées en SCD Type 1, car il s'agit d'erreurs de saisie et non d'événements métier.

Le segment client Gold/Silver/Bronze est recalculé dans l'ETL à partir du chiffre d'affaires cumulé. Pour un usage historique avancé, le changement de segment devrait être historisé en SCD Type 2. Le schéma ajoute également `date_debut`, `date_fin` et `est_actif` dans `dim_customer`.

Choix MySQL au lieu de PostgreSQL

Le cahier de charge recommande PostgreSQL pour le Data Warehouse. Dans ce rendu, MySQL est conservé de bout en bout pour rester cohérent avec le système transactionnel imposé et avec l'environnement local déjà validé. Les besoins décisionnels sont couverts avec :

- clés primaires et étrangères ;
- index sur les clés de jointure ;
- tables de reporting matérialisées par `CREATE TABLE AS SELECT` ;
- Metabase connecté directement au Data Warehouse MySQL.

Ce choix est documenté comme une hypothèse technique. Une migration PostgreSQL reste possible sans changer la logique métier du pipeline.

Pour rendre la conformité académique explicite, le dépôt fournit aussi `sql/postgres/01_create_dwh.sql`, `sql/postgres/02_check_integrity.sql` et `sql/postgres/03_reporting_materialized_views.sql`. Ces scripts implémentent les schémas PostgreSQL, les dimensions attendues, FAIT_VENTES, les index et les trois vues matérialisées natives demandées.